Lecture 13. Model Basics

Model families, fitting, predictions and residuals, formulas, interactions

Contents

13.1 Models and their purpose **2**

13.2 A simple model **2**

13.3 Visualising models **3**

13.4 Formulas and model families **4**

13.4.1 Categorical variables 4

13.4.2 Interactions 5

13.4.3 Transformations 6

13.5 Missing values and other model families **6**

13.6 Selected exercises **7**

13.1 Models and their purpose

Motivation. A model makes a pattern precise enough to measure, predict and subtract; built from scratch so lm() is no black box.

A model is a low-dimensional summary splitting data into *patterns* and *residuals*; here for exploration, not inference.

Generation versus confirmation. Exploration may reuse an observation freely; confirmation uses it once (twice means exploring). For confirmation split in advance: *training* (about 60%, explore and fit), *query* (about 20%, compare models by hand), *test* (about 20%, used once on the final model).

A model has two parts:

1. a **family of models** expressing a precise but generic pattern, e.g. the lines $y = a_1 + a_2 x$;
2. a **fitted model**, the member of the family closest to the data, e.g. $y = 3x + 7$.

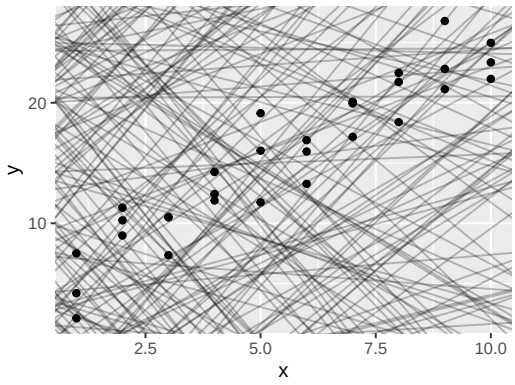
Closest is not "true": "All models are wrong, but some are useful" (G. E. P. Box).

13.2 A simple model

Motivation. Every fit needs a distance between model and data and a search minimising it.

`modelr`; `options(na.action = na.warn)` warns about dropped `NA`s. `sim1` is linear; overlay random lines:

```
models <- tibble(a1 = runif(250, -20, 40), a2 = runif(250, -5, 5))
ggplot(sim1, aes(x, y)) +
  geom_abline(aes(intercept = a1, slope = a2), data = models, alpha = 1/4) +
  geom_point()
```



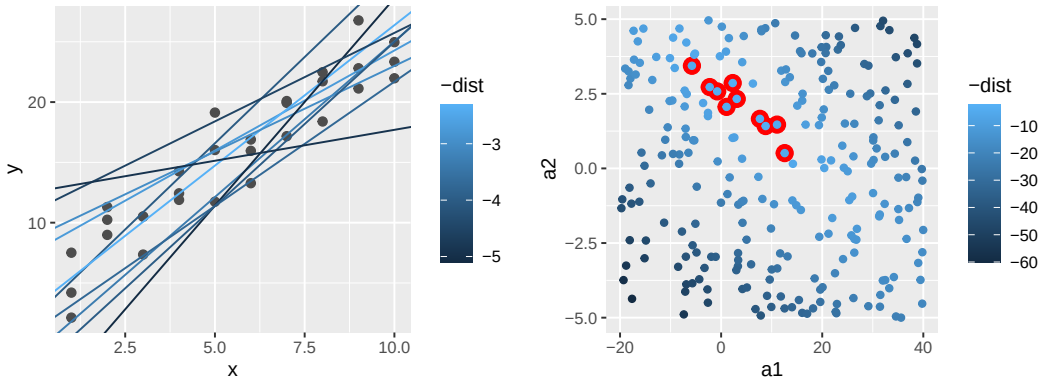
Distance: root-mean-squared vertical deviation:

```
model1 <- function(a, data) a[1] + data$x * a[2]
measure_distance <- function(mod, data) {
  diff <- data$y - model1(mod, data)
  sqrt(mean(diff ^ 2))
}
measure_distance(c(7, 1.5), sim1)
```

```
## [1] 2.665212
```

`map2_dbl()` passes scalar `a1`, `a2`, hence a helper:

```
sim1_dist <- function(a1, a2) measure_distance(c(a1, a2), sim1)
models <- models %>% mutate(dist = map2_dbl(a1, a2, sim1_dist))
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(aes(intercept = a1, slope = a2, colour = -dist),
              data = filter(models, rank(dist) <= 10))
ggplot(models, aes(a1, a2)) +
  geom_point(data = filter(models, rank(dist) <= 10), size = 4, colour = "red") +
  geom_point(aes(colour = -dist))
```



Right: models in parameter space, the ten best in red. **Grid search**: an even grid (`expand.grid()`) around the best region, repeated. Better: `optim()`, Newton–Raphson-type steps downhill:

```
best <- optim(c(0, 0), measure_distance, data = sim1)
best$par
```

```
## [1] 4.222248 2.051204
```

Linear models: `lm()` computes the closed form exactly (QR decomposition):

```
sim1_mod <- lm(y ~ x, data = sim1)
coef(sim1_mod)
```

```
## (Intercept)           x
##    4.220822    2.051533
```

Why the two agree [Supplement]

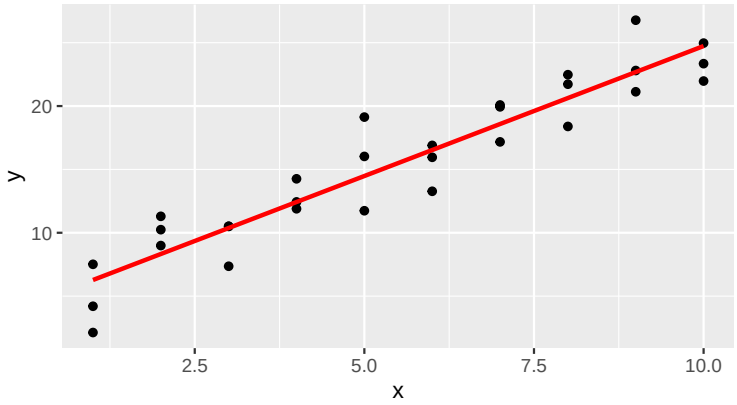
$\sqrt{\cdot}$ and $1/n$ are increasing, so this minimises $\sum_i (y_i - a_1 - a_2 x_i)^2$: least squares, $\hat{a}_2 = \sum(x_i - \bar{x})(y_i - \bar{y}) / \sum(x_i - \bar{x})^2$, $\hat{a}_1 = \bar{y} - \hat{a}_2 \bar{x}$; `optim()` approximates it.

13.3 Visualising models

Motivation. Look at predictions and residuals, for any model.

Predictions. An even grid over the data (`data_grid()`) plus `add_predictions()`:

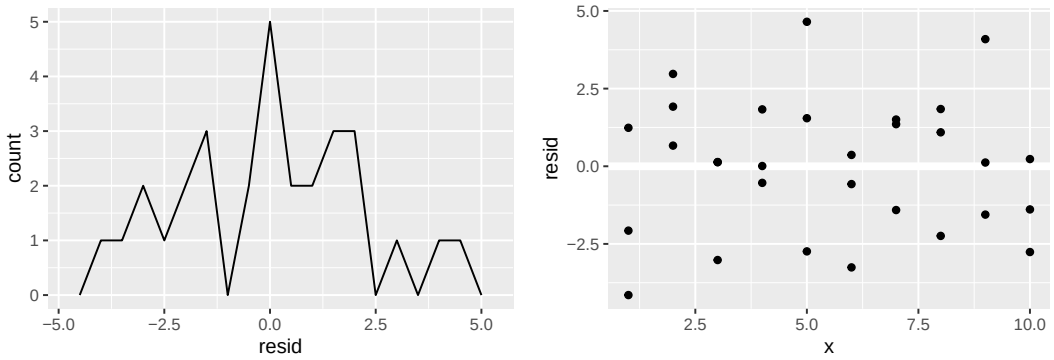
```
grid <- sim1 %>% data_grid(x) %>% add_predictions(sim1_mod)
ggplot(sim1, aes(x)) +
  geom_point(aes(y = y)) +
  geom_line(aes(y = pred), data = grid, colour = "red", linewidth = 1)
```



This shows what any model captured.

Residuals (observed − predicted) show what it missed; `add_residuals()` needs the observed data, not a grid. If the pattern is captured, they look like noise:

```
sim1 <- sim1 %>% add_residuals(sim1_mod)
ggplot(sim1, aes(resid)) + geom_freqpoly(binwidth = 0.5)
ggplot(sim1, aes(x, resid)) + geom_ref_line(h = 0) + geom_point()
```



13.4 Formulas and model families

Motivation. A formula becomes a design matrix, which fixes what a model can fit.

`y ~ x` means $y = a_1 + a_2 x$; `model_matrix()` shows the design matrix, one column per coefficient, intercept of ones unless `- 1`.

```
df <- tribble(~y, ~x1, ~x2,  4, 2, 5,  5, 1, 6)
model_matrix(df, y ~ x1 + x2)
```

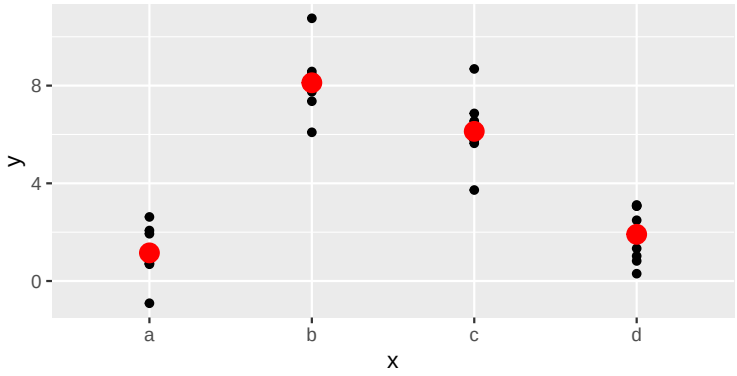
```
## # A tibble: 2 x 3
##   `(Intercept)`    x1    x2
##           <dbl> <dbl> <dbl>
## 1             1     2     5
## 2             1     1     6
```

13.4.1 Categorical variables

`y ~ sex` gives an indicator `sexmale` (`sexfemale` would be redundant, one minus it). A categorical `x` predicts each category's mean:

```
mod2 <- lm(y ~ x, data = sim2)
```

```
grid <- sim2 %>% data_grid(x) %>% add_predictions(mod2)
ggplot(sim2, aes(x)) +
  geom_point(aes(y = y)) +
  geom_point(data = grid, aes(y = pred), colour = "red", size = 4)
```

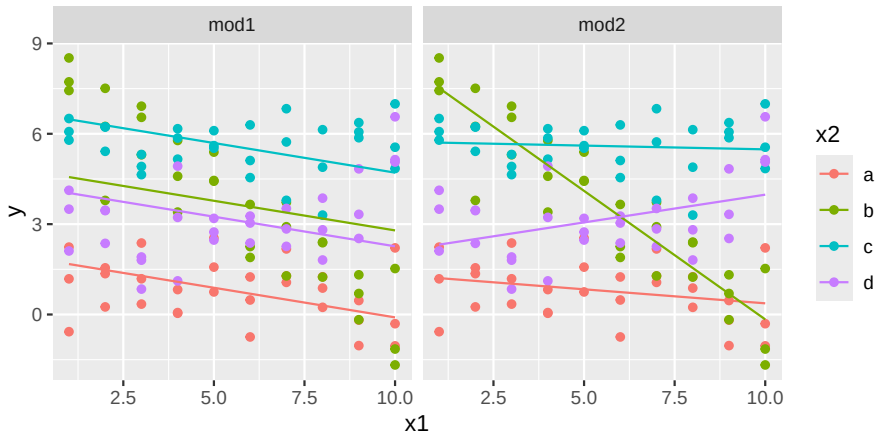


An unseen level (`tibble(x = "e")`) is an error.

13.4.2 Interactions

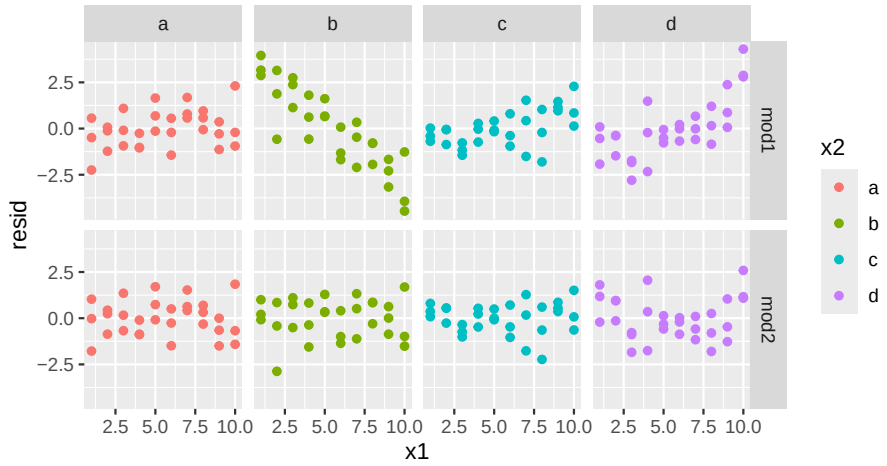
`sim3`: continuous `x1`, categorical `x2`. `y ~ x1 + x2`: parallel lines; `y ~ x1 * x2` adds `x1:x2`, a slope per group. `gather_predictions()` stacks models:

```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3)
grid <- sim3 %>% data_grid(x1, x2) %>% gather_predictions(mod1, mod2)
ggplot(sim3, aes(x1, y, colour = x2)) +
  geom_point() +
  geom_line(data = grid, aes(y = pred)) +
  facet_wrap(~ model)
```



`mod1`'s residuals show patterns in `b`, `c`, `d`; `mod2`'s look random (formal comparison: inference).

```
sim3 <- sim3 %>% gather_residuals(mod1, mod2)
ggplot(sim3, aes(x1, resid, colour = x2)) +
  geom_point() +
  facet_grid(model ~ x2)
```



Two continuous (`sim4`): `y ~ x1 * x2` is $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$; on a grid from `seq_range(x1, 5)` the interaction shows as slopes changing with the other variable.

13.4.3 Transformations

In the formula: `log(y) ~ sqrt(x1) + x2`. Formula operators (`+`, `*`, `^`, `-`) need `I()`: `y ~ x + I(x^2)` is $a_1 + a_2 x + a_3 x^2$, but `y ~ x^2 + x` is `y ~ x`. Linear in the coefficients suffices for curves. `poly(x, 2)`: orthogonal polynomials; `splines::ns(x, df)`: natural splines, tamer than polynomials outside the data.

```
df <- tribble(~y, ~x,  1, 1,  2, 2,  3, 3)
model_matrix(df, y ~ x + I(x^2))
```

```
## # A tibble: 3 x 3
##   `(Intercept)`     x `I(x^2)`
##           <dbl> <dbl>    <dbl>
## 1             1     1        1
## 2             1     2        4
## 3             1     3        9
```

Extrapolation [Supplement]

Outside the data range only the family decides (polynomial to $\pm\infty$, spline linear); neither is evidence, so do not trust far extrapolation.

13.5 Missing values and other model families

`lm()` drops rows with `NA` (silently, or warning with `na.warn`); `nobs(mod)` counts those used.

Linear models assume continuous residuals of constant variance. Extensions:

- `glm()`: binary or count responses;
- `mgcv::gam()`: smooth `y ~ s(x)`;
- penalised (`glmnet`): shrink coefficients;
- robust `MASS::rlm()`: less outlier influence;
- trees and forests (`rpart`, `randomForest`, `xgboost`).

All are explored by predictions and residuals.

13.6 Selected exercises

Motivation. Parameterisations of the same fitted values, and residuals orthogonal to the design.

Exercise 13.1 (R4DS Exercise 23.2.1.3)

What is the problem with optimising the three-parameter model $f(x; a) = a_1 + a_2 x + a_3$ by minimising the root-mean-squared distance?

Solution.

Step 1. Predictions depend on a only via $(a_1 + a_3, a_2)$, so $d(a_1 + t, a_2, a_3 - t) = d(a)$ for all t.

Step 2. With $c = a_1 + a_3$, the minimum over (c, a_2) is the least-squares line ("Why the two agree"), unique for non-constant x; the minimisers of d form the line $\{(c - t, \hat{a}_2, t) : t \in \mathbb{R}\}$: not *identifiable*.

Step 3. `optim()` stops at a start-dependent point of it, reporting different a_1, a_3 for the same line. The parameterisation is redundant; formulas never create two intercepts. ■

Exercise 13.2 (R4DS Exercise 23.4.5.1)

For `sim2`, whose predictor `x` has levels a, b, c, d, compare `y ~ x` with `y ~ x - 1`. What happens to the model equation and to the predictions?

Solution.

Step 1. (Equations.) With intercept: columns $1, x_b, x_c, x_d$, $\hat{y} = a_1 + a_2 x_b + a_3 x_c + a_4 x_d$. Without: all four indicators, $\hat{y} = a_a x_a + a_b x_b + a_c x_c + a_d x_d$.

Step 2. (Same column space.) $x_a + x_b + x_c + x_d = 1$ and $x_a = 1 - x_b - x_c - x_d$, so both designs span the same space; least squares projects y onto it, so fitted values, residuals and predictions coincide.

Step 3. (Coefficients.) Without intercept $\hat{a}_\ell$ is group ℓ's mean; with it $\hat{a}_1$ is group a's mean and $\hat{a}_2, \hat{a}_3, \hat{a}_4$ differences from it. ■

Exercise 13.3 (R4DS Exercise 23.4.5.2)

Which columns does `model_matrix()` create for `y ~ x1 * x2` in `sim3` (continuous `x1`, `x2` with levels a, b, c, d) and in `sim4` (both continuous)? Why is `*` a good shorthand?

Solution.

Step 1. `x1 * x2` = `x1 + x2 + x1:x2`. In `sim3`: intercept, x_1, x_{2b}, x_{2c}, x_{2d}, $x_1 x_{2b}$, $x_1 x_{2c}$, $x_1 x_{2d}$, eight columns: an intercept and a slope per group.

Step 2. In `sim4`: intercept, x_1, x_2, $x_1 x_2$, so $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$ with slope $a_1 + a_{12} x_2$ in x_1.

Step 3. Without main effects the slope in x_1 would vanish at $x_2 = 0$ (or the reference group), depending on the arbitrary origin or coding of x_2; `*` always includes the lower-order terms. ■

Exercise 13.4 (what a residual plot can show)

Let $\hat{a}$ be the least-squares fit of a model whose design matrix X contains an intercept column and a column x, and let $r = y - X\hat{a}$. Show that the residuals have mean 0 and zero sample correlation with x (and with every other column of X). What does this imply for a plot of r against x, and against a variable not in the model?

Solution [Supplement].

Step 1. $\min_a \|y - Xa\|^2$ has gradient $-2X^\mathsf{T}(y - Xa) = 0$ at the minimiser: $X^\mathsf{T} r = 0$.

Step 2. The intercept column gives $\sum_i r_i = 0$, $\bar{r} = 0$; then $\sum_i (x_i - \bar{x})(r_i - \bar{r}) = \sum_i x_i r_i - \bar{x} \sum_i r_i = 0$: zero covariance and correlation.

Step 3. So against an included x there is no linear trend; any pattern (curvature, spread, clusters) is beyond the linear term. Against an omitted variable any correlation is possible, and a trend says: add it. Thus `mod1`'s residuals by group revealed the missing $x_1 x_{2b}, x_1 x_{2c}, x_1 x_{2d}$. ■